PROJECT ON HETEROGENEOUS COMPUTING

COURSE: EFFICIENT HETEROGENEOUS COMPUTING, M.EIC033
MASTER IN INFORMATICS AND COMPUTING ENGINEERING (M.EIC)
FACULTY OF ENGINEERING OF THE UNIVERSITY OF PORTO (FEUP)

October 2026

This document describes the main project work to be done for the "Efficient Heterogeneous Computing" course of the Master in Informatics and Computing Engineering (M.EIC), of the Faculty of Engineering of the University of Porto (FEUP).

THE PROGRAM

The k-Nearest Neighbors (*kNN*) [1] is a widely used machine learning technique for classification and regression[1]. For classification, and using the basic version of *kNN*, the training corresponds to the storage of the *n* training instances, each one represented as a vector of d features (dimensions), with each vector labeled by the respective class. For each instance to classify, *kNN* determines the *k* nearest instances, i.e., the *k* training instances nearest the instance and then classifies based on the classes of those *k* instances. One possibility is to classify with the class most frequent in the k closest training instances.

The code of the project implements *kNN* in C targeting the classification of ***m*** instances. All instances consist of vectors of ***d*** features represented in single- or double-precision floating point numbers (float or double types in C). Training considers ***c*** classes, and the Euclidean distance is used for the distance between the instance to classify and each of the training instances.

The configuration of a *kNN* implementation is defined by the tuple (*m, k, n, d, c, t*), where ***m*** represents the number of instances to classify, ***k*** the number of nearest neighbors to consider, ***n*** the number of training instances (stored in the *kNN* database), ***d*** is the number of features, ***c*** is the number of classes, and ***t*** the data type used for the data and distance calculations.

The computational complexity of the basic *kNN* is *O(nd + kn)*.

GOALS

The goal is to accelerate the execution of the function *knn_classifyinstance* in two environments: (1) a PC/desktop; (2) an embedded system with a hardware accelerator. For parallelization targeting a multicore CPU, the intention is to use the OpenMP directive-driven programming model. For optimizations regarding the generation of the hardware accelerator, the intention is to use the optimization directives provided by the specific -HLS (High-Level Synthesis) compilers.

The implementation of this function and of its auxiliary functions is in the file `knn.c`. However, changes do not need to be limited to the function *knn_classifyinstance*.-

We will use two scenarios for the experiments in the desktop and in the embedded device with the hardware accelerator. In all of them, data are embedded in the code of the program (i.e., data structures are initialized by static data provided in the code), and the program does not receive any arguments from the command line. The

[1] A brief description of the kNN technique is presented in https://en.wikipedia.org/wiki/K-neare_st_neighbors_algorithm.

inputs data for scenarios inputs are from WISDM[2] v1.1 [2], and from PAMAP2, two datasets of Human Activity Recognition (HAR), with data from sensors in mobile devices and collected by different users. These two scenarios are used to acquire profiling data when the number of training, testing instances, and features is larger. They can also be used to evaluate the impact of optimizations and of the final implementations.

In those scenarios, the *kNN* implementations to be improved are based on the following parameters:

Benchmark	**K**	**Dataset**	**#classes (C)**	**#features (d)**	**Training instances (n)**	**Testing instances (m)**	**Data type used (DT)**
kNN	3, 5, 20	WISDM	6	43	4336	1082	float or double
		PAMAP2	22	90	6186	1008	float or double

The 6 classes (human activities) of the WISDM dataset are represented here using the following numerical identifiers (IDs):

Class: Upstairs ID = 3
Class: Jogging ID = 0
Class: Sitting ID = 2
Class: Standing ID = 1
Class: Walking ID = 4
Class: Downstairs ID = 5

The program can be compiled using the Makefile provided.

Below is part of the output from executing the code with the WISDM dataset, k=20, and double-precision floating-point.

```
> .\knn
Array data points initialized with data in files ...
Executing kNN...
class id: 4
…
class id: 4
class id: 4
class id: 4
kNN done.

HAR: number of features = 43
HAR: number of classes = 6
kNN: k = 3
kNN: number of training instances = 7383
kNN: number of testing instances = 3163
kNN: data type used = double
kNN: number of classified instances = 3163
kNN: number of classifications wrong = 791
kNN: number of classifications right = 2372
```

[2] This data has been released by the Wireless Sensor Data Mining (WISDM) Lab: http://www.cis.fordham.edu/wisdm/

```
kNN: accuracy = 74.99 %

Total execution time:  0.6872 s
Inference time: 0.2173 ms
Throughput: 4602.88 inferences/s
```

At the end of execution, the program reports whether the produced outputs are correct or not, and the accuracy[3] achieved in the classification of the input instances. It also prints the computation time taken to classify the points in the input testing data.

STEPS TO BE FOLLOWED AND REPORTED:

1. Analysis and improvements targeting Intel/AMD multicore CPUs in a PC/desktop/laptop machine. **[checkpoint 1]**
   - 1.1. Analyze via profiling (*gprof* and *gcov*) and via Intel VTune and Intel Advisor the program given. For example, identify the hotspots, draw the call graph and the task graph of the program, and decorate them with the information you find useful for helping to improve the execution time and the selection of the code regions to be possibly migrated to a hardware accelerator.
   - 1.2. Characterize the software code versions on the local machine. Use the *gcc* compiler and selected flags and show the energy consumption, power dissipation, and execution time gains that you could obtain with the different versions over the original code provided.
   - 1.3. Considering its execution on a PC/desktop/laptop machine, try to improve its overall performance (it can also include parallelization via OpenMP) and use Intel VTune and Intel Advisor to help you (e.g., verify the positions in the Roofline model of the subsequent versions).
2. Analysis of the baseline implementation and targeting a SoC with a multicore ARM CPU and hardware accelerators using FPGAs. **[checkpoint 2]**
   - 2.1. Analyze the code and evaluate possible alternatives to offload computations to the target FPGA. Characterize and optimize the select code for offloading (use Vitis HLS).
     - 2.1.1. Report the FPGA resources used, the maximum clock frequency achieved, and the latency (in number of clock cycles)
     - 2.1.2. Report the execution times using the SW version (ARM CPU) and the HW/SW version (ARM CPU + FPGA accelerator)
   - 2.2. Evaluate the hardware/software system and the overall speedup obtained (i.e., CPU + accelerator vs. CPU).
3. Improvements and analysis of optimizations for the main kNN function in the FPGA. **[checkpoint 3]**
   - 3.1. Report the selected versions you may want to evaluate
   - 3.2. Analyze the optimizations and discuss possible extensions
   - 3.3. Compare the results of the software version executed in the CPU with the hardware/software version (CPU + FPGA accelerator)

Note 1: for step 1, use the two scenarios presented before.

Note 2: for steps 2 and 3, use the following scenario:

[3] We note that the use case used has not involved feature selection to improve the kNN accuracy.

Benchmark	K	Dataset	#classes (C)	#features (d)	Training instances (n)	Testing instances (m)	Data type used (DT)
kNN	3	WISDM	6	43	4336	1082	float

NOTES REGARDING THE WORK AND/OR THE TECHNICAL REPORT:

The most important experiments, e.g., involving compiler optimizations and/or code transformations, and the respective results shall be reported even if they do not improve execution time, power dissipation, and/or energy consumption.

In addition, the report shall describe:

- The experimental setup (e.g., the machine used where the experiments were conducted) shall be described in detail;
- The results achieved and an analysis of those results;
- Possible code transformations and optimizations that you think can have a significant impact, but did not have the time to evaluate.

REFERENCES

[1] T. Cover and P. Hart, "Nearest neighbor pattern classification," in IEEE Transactions on Information Theory, vol. 13, no. 1, pp. 21-27, January 1967, doi: https://doi.org/10.1109/TIT.1967.1053964.

[2] Jennifer R. Kwapisz, Gary M. Weiss, and Samuel A. Moore. 2011. Activity recognition using cell phone accelerometers. SIGKDD Explor. Newsl. 12, 2 (December 2010), 74–82. doi: https://doi.org/10.1145/1964897.1964918.